

Epic 5: Application Building

Story 3: Application Testing and Validation

After integrating the trained XGBoost model with the Flask web application, the system is tested to verify its functionality and prediction accuracy. The application is executed locally, and different weather parameters are provided to ensure that the model generates correct flood risk predictions.

Launching the Application

The Flask application is started using the following steps:

1. Open Anaconda Prompt or the command prompt.
2. Navigate to the project directory containing the app.py file.
3. Execute the following command:

```
python app.py
```

After execution, the Flask development server starts successfully.

Accessing the Web Application

The terminal displays a local server URL, typically:

```
http://127.0.0.1:5000/
```

This URL is opened in a web browser to access the AI Flood Prediction System.

Prediction Workflow

The user interacts with the application through the following steps:

1. Open the Home Page.
2. Click the Predict Flood button.
3. Enter the required weather parameters such as rainfall, temperature, humidity, cloud cover, and other relevant inputs.
4. Click the Submit button to send the data to the Flask backend.

Backend Processing

After receiving the user input, the Flask application performs the following operations:

- Collects the weather parameters submitted through the web form.
- Applies the saved StandardScaler to standardize the input values.
- Loads the saved XGBoost model.

- Generates the flood prediction using the trained model.
- Displays the prediction result to the user.

Result Display

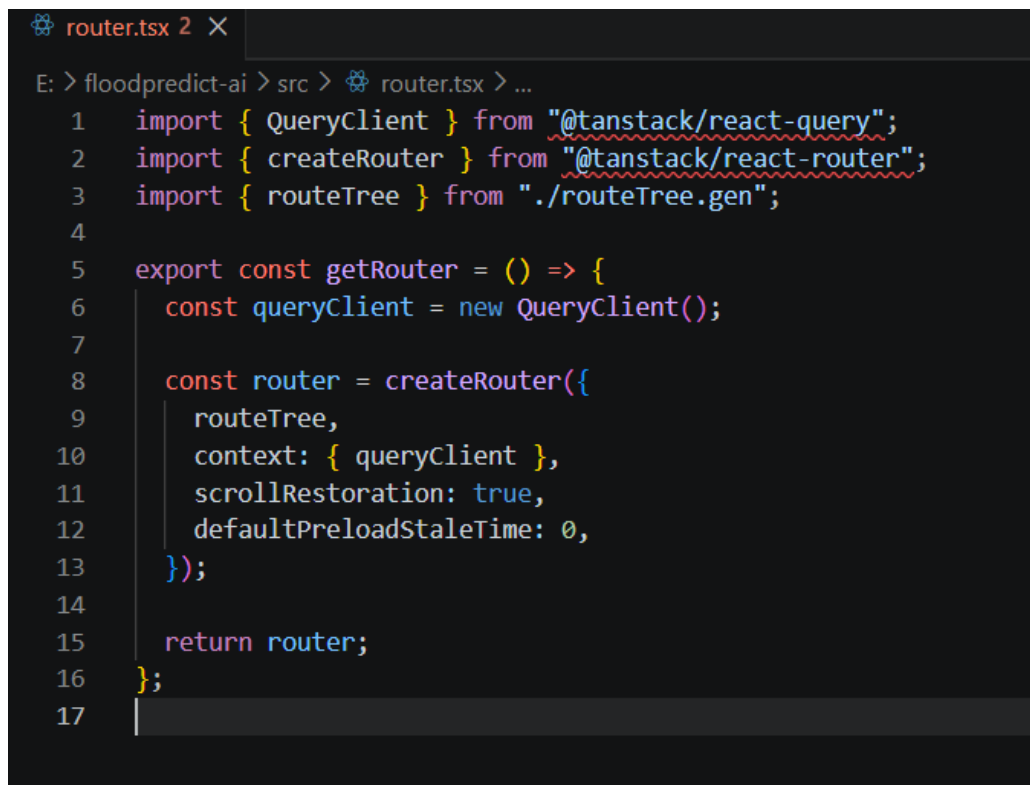
Based on the prediction generated by the model, the application redirects the user to the appropriate result page:

- `chance.html` – Displays a Flood Risk Alert when flooding is predicted.
- `no_chance.html` – Displays a Safe Status message when no flood risk is detected.

Outcome

- Successfully launched the Flask application.
- Verified that all web pages and routes function correctly.
- Validated the integration of the XGBoost model with the Flask backend.
- Confirmed that the application provides accurate real-time flood predictions based on user input.
- Ensured a smooth and user-friendly prediction workflow from input submission to result display.

Figure 20: Testing the AI Flood Prediction System by entering weather parameters and displaying the final flood prediction result.



```

router.tsx 2 X
E: > floodpredict-ai > src > router.tsx > ...
1  import { QueryClient } from "@tanstack/react-query";
2  import { createRouter } from "@tanstack/react-router";
3  import { routeTree } from "./routeTree.gen";
4
5  export const getRouter = () => {
6    const queryClient = new QueryClient();
7
8    const router = createRouter({
9      routeTree,
10     context: { queryClient },
11     scrollRestoration: true,
12     defaultPreloadStaleTime: 0,
13   });
14
15   return router;
16 };
17

```

